CLOUD & VIRTUALISATION | TEK-UP UNIVERSITY

Lab 03

Introduction to Docker on Red Hat Enterprise Linux

Student

Assil Mhadhbi

Class

Ing1, Groupe C (evening course)

Date

17 October 2026

Virtual machine	VM1-RedHat-Desktop (VirtualBox)
Operating system	Red Hat Enterprise Linux 10 (the lab was written for RHEL 9)
Docker version	Docker Engine – Community 29.9.1

1. Objective

The goal of this lab is to become familiar with the Docker containerization platform on a Red Hat Enterprise Linux virtual machine. The work covers three parts: installing Docker, managing images and containers, and running a simple Hello World container.

2. Environment

Item	Value
Hypervisor	Oracle VirtualBox
Guest OS	Red Hat Enterprise Linux 10 (64-bit)
Kernel	6.12.0-211.65.1.el10_2.x86_64
Processors	2 vCPU
Memory	4096 MB
Network	NAT (Intel PRO/1000 MT Desktop), Internet access confirmed with ping
Subscription	Registered (BaseOS and AppStream repositories available)
User	darkdiva, with sudo rights (group wheel)

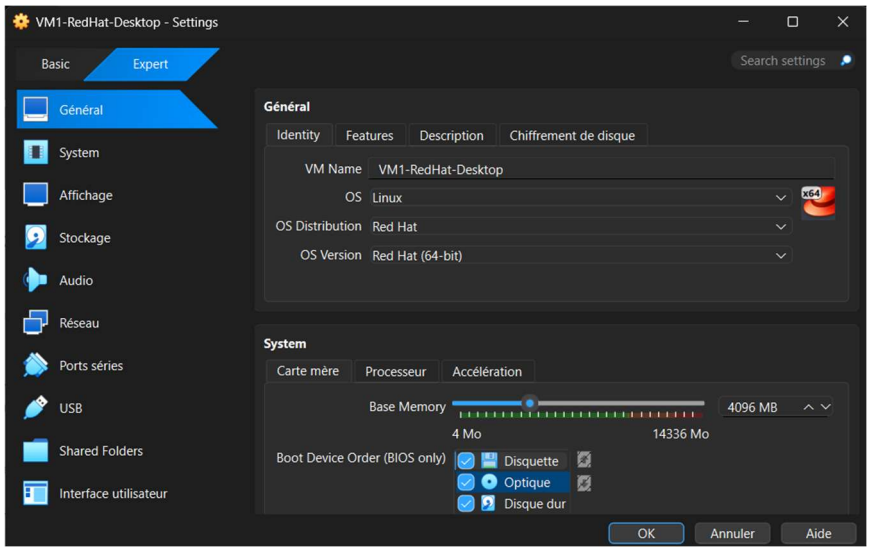


Figure 1 – VM general settings and memory (4096 MB)

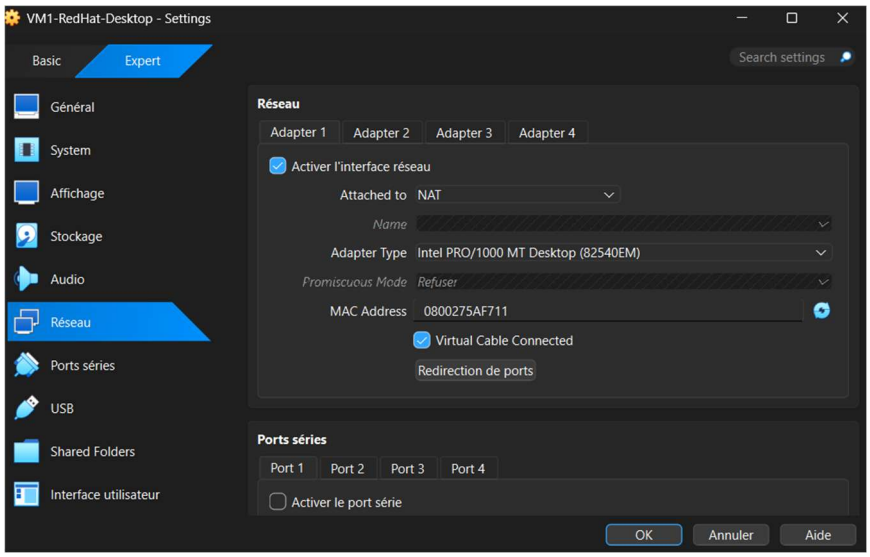


Figure 2 – VM network adapter attached to NAT

Checks before the installation. The kernel version, the subscription status and the repositories were verified, and the Internet connection was tested with ping (0% packet loss).

```
darkdiva@localhost:~
darkdiva@localhost:~$ uname -r
6.12.0-211.65.1.el10_2.x86_64
darkdiva@localhost:~$ sudo subscription-manager status
[sudo] password for darkdiva:
+-------------------------------------------+
   System Status Details
+-------------------------------------------+
Overall Status: Registered

darkdiva@localhost:~$ sudo dnf repolist
Updating Subscription Management repositories.
repo id                              repo name
rhel-10-for-x86_64-appstream-rpms Red Hat Enterprise Linux 10 fo
r x86_64 - AppStream (RPMs)
rhel-10-for-x86_64-baseos-rpms    Red Hat Enterprise Linux 10 fo
r x86_64 - BaseOS (RPMs)
darkdiva@localhost:~$ ping -c 2 google.com
PING google.com (172.217.18.110) 56(84) bytes of data.
64 bytes from ncmrsb-aj-in-f14.1e100.net (172.217.18.110): icmp_
seq=1 ttl=64 time=31.0 ms
64 bytes from ncmrsb-aj-in-f14.1e100.net (172.217.18.110): icmp_
seq=2 ttl=64 time=29.1 ms

--- google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 29.132/30.072/31.013/0.940 ms
darkdiva@localhost:~$
```

Figure 3 – Kernel version, subscription status, repositories and connectivity test

Note on the version. The VM runs RHEL 10, whereas the lab was written for RHEL 9. The commands are the same, and the only visible difference is the kernel tag (el10 instead of el9).

3. Installing Docker CE

Docker CE (Method 1 of the lab) was installed from the official Docker repository. Method 2 (Podman) was not used.

3.1 Removing conflicting packages

Podman and its dependencies, which come with RHEL and conflict with Docker CE, were removed first.

```
sudo dnf remove docker docker-client docker-client-latest docker-common \
  docker-latest docker-latest-logrotate docker-logrotate docker-selinux \
  docker-engine-selinux docker-engine podman runc
```

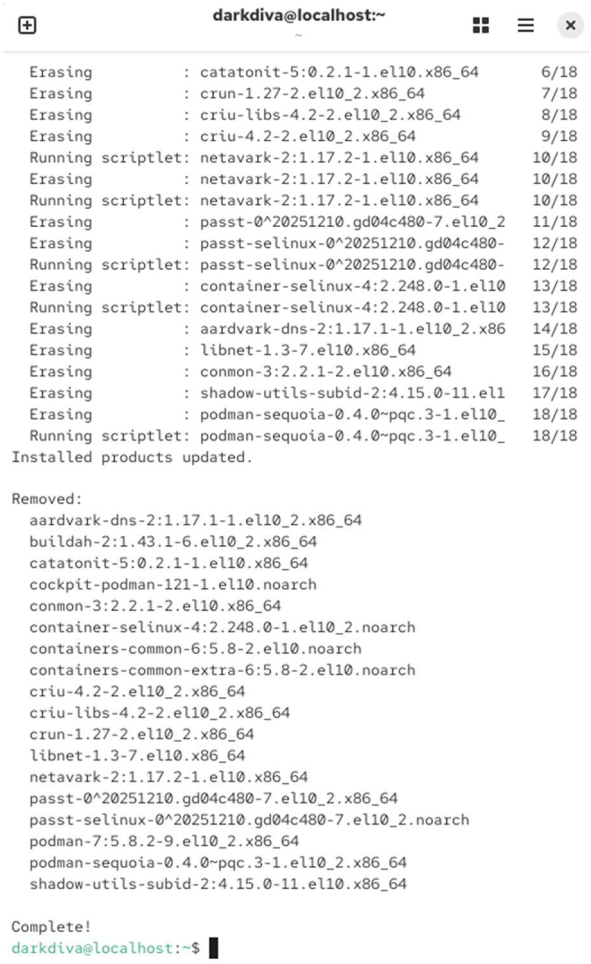


Figure 4 – Podman, buildah and related packages removed

3.2 Adding the Docker repository and installing

```
sudo dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

The repository docker-ce-stable was added and the Docker GPG key was imported. Seven packages were installed, plus container-selinux as a dependency from the Red Hat repositories (SELinux stays in Enforcing mode).

```
darkdiva@localhost:~$ sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-
plugin
[sudo] password for darkdiva:
Updating Subscription Management repositories.
Last metadata expiration check: 0:04:01 ago on Sat 17 Oct 2026 04:52:30 PM CET.
Dependencies resolved.
================================================================
 Package       Arch   Version            Repository        Size
================================================================
Installing:
 containerd.io x86_64 2.4.2-2.el10       docker-ce-stable  35 M
 docker-buildx-plugin
               x86_64 0.38.0-1.el10      docker-ce-stable  18 M
 docker-ce     x86_64 3:29.9.1-1.el10    docker-ce-stable  26 M
 docker-ce-cli x86_64 1:29.9.1-1.el10    docker-ce-stable 8.9 M
 docker-compose-plugin
               x86_64 5.6.0-1.el10       docker-ce-stable 8.4 M
Installing dependencies:
 container-selinux
               noarch 4:2.248.0-1.el10_2 rhel-10-for-x86_64-appstream-rpms
                                                          66 k
Installing weak dependencies:
 docker-ce-rootless-extras
               x86_64 29.9.1-1.el10      docker-ce-stable 5.6 M

Transaction Summary
================================================================
Install  7 Packages

Total download size: 102 M
Installed size: 387 M
Downloading Packages:
(1/7): docker-buildx-plugin-0.3 3.5 MB/s |  18 MB     00:05
(2/7): docker-ce-29.9.1-1.el10. 2.0 MB/s |  26 MB     00:12
(3/7): docker-ce-cli-29.9.1-1.e 941 kB/s | 8.9 MB     00:09
(4/7): containerd.io-2.4.2-2.el 2.1 MB/s |  35 MB     00:16
(5/7): container-selinux-2.248. 117 kB/s |  66 kB     00:00
```

Figure 5 – Installation transaction (docker-ce 29.9.1, containerd.io 2.4.2)

```
(6/7): docker-ce-rootless-extra 1.3 MB/s | 5.6 MB     00:04
(7/7): docker-compose-plugin-5. 2.4 MB/s | 8.4 MB     00:03
----------------------------------------------------------------
Total                           5.5 MB/s | 102 MB     00:18
Docker CE Stable - x86_64       9.7 kB/s | 1.6 kB     00:00
Importing GPG key 0x621E9F35:
 Userid     : "Docker Release (CE rpm) <docker@docker.com>"
 Fingerprint: 060A 61C5 1B55 8A7F 742B 77AA C52F EB6B 621E 9F35
 From       : https://download.docker.com/linux/rhel/gpg
Key imported successfully
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                        1/1
  Running scriptlet: container-selinux-4:2.248.0-1.el10_2   1/7
  Installing       : container-selinux-4:2.248.0-1.el10_2   1/7
  Running scriptlet: container-selinux-4:2.248.0-1.el10_2   1/7
  Installing       : docker-buildx-plugin-0.38.0-1.el10.x   2/7
  Running scriptlet: docker-buildx-plugin-0.38.0-1.el10.x   2/7
  Installing       : docker-compose-plugin-5.6.0-1.el10.x   3/7
  Running scriptlet: docker-compose-plugin-5.6.0-1.el10.x   3/7
  Installing       : docker-ce-cli-1:29.9.1-1.el10.x86_64   4/7
  Running scriptlet: docker-ce-cli-1:29.9.1-1.el10.x86_64   4/7
  Installing       : containerd.io-2.4.2-2.el10.x86_64      5/7
  Running scriptlet: containerd.io-2.4.2-2.el10.x86_64      5/7
  Installing       : docker-ce-rootless-extras-29.9.1-1.e   6/7
  Running scriptlet: docker-ce-rootless-extras-29.9.1-1.e   6/7
  Installing       : docker-ce-3:29.9.1-1.el10.x86_64       7/7
  Running scriptlet: docker-ce-3:29.9.1-1.el10.x86_64       7/7
  Running scriptlet: container-selinux-4:2.248.0-1.el10_2   7/7
  Running scriptlet: docker-ce-3:29.9.1-1.el10.x86_64       7/7
Installed products updated.

Installed:
  container-selinux-4:2.248.0-1.el10_2.noarch
  containerd.io-2.4.2-2.el10.x86_64
  docker-buildx-plugin-0.38.0-1.el10.x86_64
  docker-ce-3:29.9.1-1.el10.x86_64
```

Figure 6 – GPG key imported and installation completed

3.3 Starting the service

```
sudo systemctl enable --now docker
sudo systemctl status docker
```

The service is active (running) and enabled at boot. The preset is disabled on RHEL, which is why the enable option is needed.

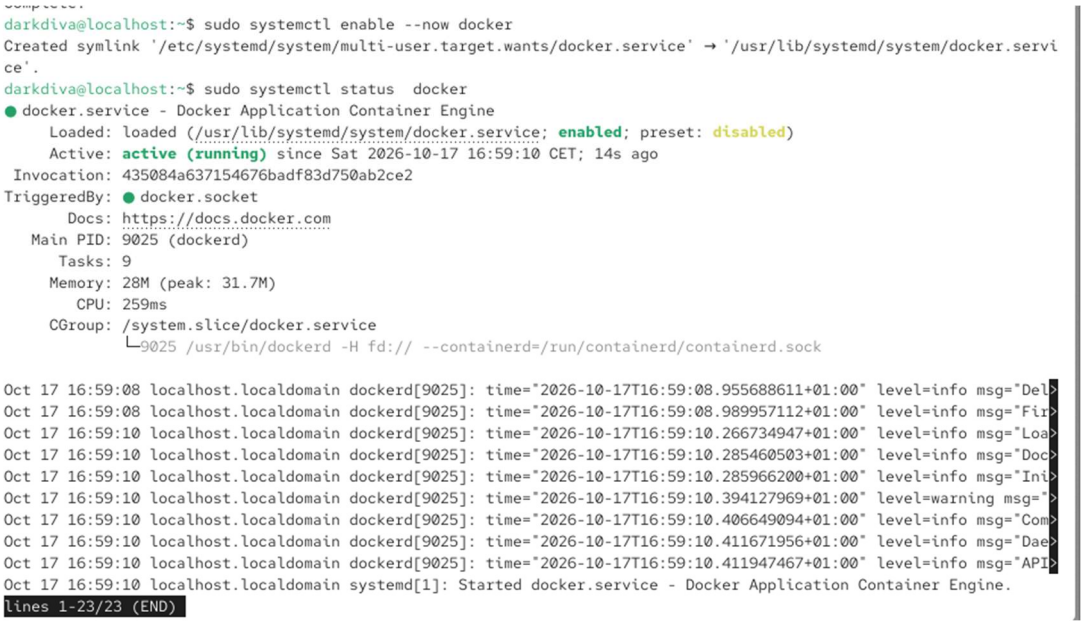


Figure 7 – docker.service active (running) with dockerd as main process

3.4 Docker privileges

The docker group already existed (GID 976). The user was added to it, then the session was reopened so that the change would apply.

```
getent group docker
sudo usermod -aG docker $USER
```



Figure 8 – Docker group checked and user added (before reopening the session)

After logging back in, id -nG lists docker, and docker works without sudo. Client and server are both version 29.9.1, and cgroup v2 is in use.



Figure 9 – id -nG, docker version and docker info

Remark on the storage driver. The lab expects overlay2, but this installation shows overlayfs. Recent Docker versions (29 and later) use the containerd image store by default, and overlayfs is its overlay-based driver, so this difference is normal.

4. Image management

4.1 Listing and searching images

docker images returned an empty list on a fresh installation. docker search centos lists the official centos image (marked DEPRECATED, 7780 stars) and many community images. The filter --filter=stars=5 keeps only images with at least 5 stars.



Figure 10 – docker images (empty) and docker search centos

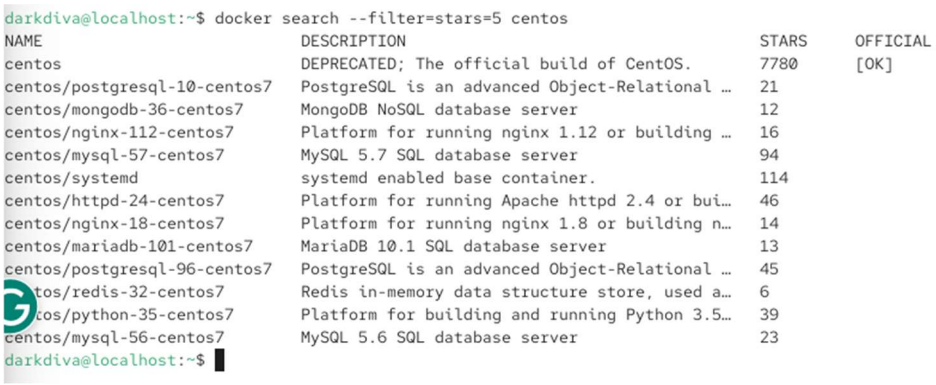


Figure 11 – docker search --filter=stars=5 centos

4.2 Pulling an image: the failing command

```
docker pull centos
```

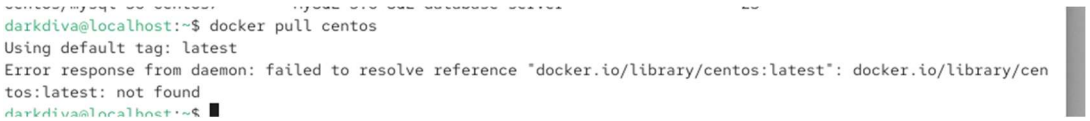


Figure 12 – Failed pull of centos:latest

Question 1: what error message is displayed?

Error response from daemon: failed to resolve reference "docker.io/library/centos:latest": docker.io/library/centos:latest: not found.

Question 2: why does this error occur?

When no tag is given, Docker uses the default tag latest. The tag centos:latest no longer exists on Docker Hub: CentOS Linux reached end of life, the official image was marked DEPRECATED (as the search result shows) and its latest tag was removed. Docker therefore cannot find the requested manifest.

Question 3: which version can be used instead?

An explicit tag that still exists must be given, for example centos:centos7 (docker pull centos:centos7).

4.3 Successful pulls and bonus: UBI9

```
docker pull centos:centos7
docker pull registry.access.redhat.com/ubi9/ubi
docker images
```

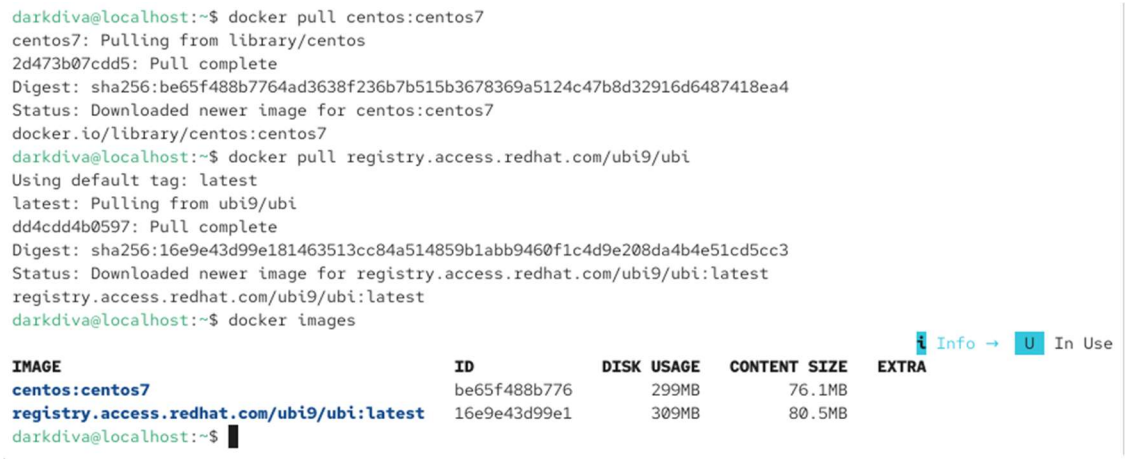


Figure 13 – Pull of centos:centos7 and ubi9/ubi, then docker images

Image	Disk usage	Content size (compressed)
centos:centos7	299 MB	76.1 MB
registry.access.redhat.com/ubi9/ubi	309 MB	80.5 MB

The two images have similar sizes. UBI9 is about 10 MB larger on disk because its packages are more recent. Unlike CentOS 7, UBI is Red Hat's base image that is still maintained and freely redistributable.

4.4 Removing an image

```
docker rmi registry.access.redhat.com/ubi9/ubi
```

The tag is first untagged and then the image is deleted. The image was downloaded again afterwards to keep it for the comparison. The command docker rmi $(docker images -a -q), which deletes all images, was not run, because the images were needed for the next parts.

```
darkdiva@localhost:~$ docker rmi registry.access.redhat.com/ubi9/ubi
Untagged: registry.access.redhat.com/ubi9/ubi:latest
Deleted: sha256:16e9e43d99e181463513cc84a514859b1abb9460f1c4d9e208da4b4e51cd5cc3
darkdiva@localhost:~$ docker images
```

i Info → U In Use

```
IMAGE           ID             DISK USAGE   CONTENT SIZE   EXTRA
centos:centos7  be65f488b776         299MB         76.1MB
darkdiva@localhost:~$ docker pull registry.access.redhat.com/ubi9/ubi
Using default tag: latest
latest: Pulling from ubi9/ubi
dd4cdd4b0597: Pull complete
Digest: sha256:16e9e43d99e181463513cc84a514859b1abb9460f1c4d9e208da4b4e51cd5cc3
Status: Downloaded newer image for registry.access.redhat.com/ubi9/ubi:latest
registry.access.redhat.com/ubi9/ubi:latest
darkdiva@localhost:~$
```

Figure 14 – docker rmi, docker images and a new pull of ubi9/ubi

5. Container management

The following sequence illustrates the life cycle of a container.

Step	Command	Observed result
Create	docker create --name MyFirstContainer centos:centos7	Container ID returned; docker ps is empty
List all	docker ps -a	Status Created, command /bin/bash
Start	docker start MyFirstContainer	Status Exited (0) a few seconds later
Stop	docker stop MyFirstContainer	Nothing to stop, container had already exited
Create and start	docker run centos:centos7	New container with random name (goofy_pike), Exited (0)
Remove one	docker rm MyFirstContainer	Only goofy_pike remains
Remove all	docker rm -f $(docker ps -a -q)	Empty list
Auto-remove	docker run --rm centos:centos7	docker ps -a is empty: nothing is kept



Figure 15 – Container life cycle

Why do the containers exit immediately? The default command of the centos:centos7 image is /bin/bash. Without an interactive terminal (option -it), bash receives no input and ends at once. A container only lives as long as its main process. The exit code 0 shows a normal end without error. The option --rm deletes the container automatically when it exits.

6. Running a command in a container: shared kernel

```
docker run --rm ubuntu uname -a
docker run --rm centos:centos7 uname -a
uname -a
```

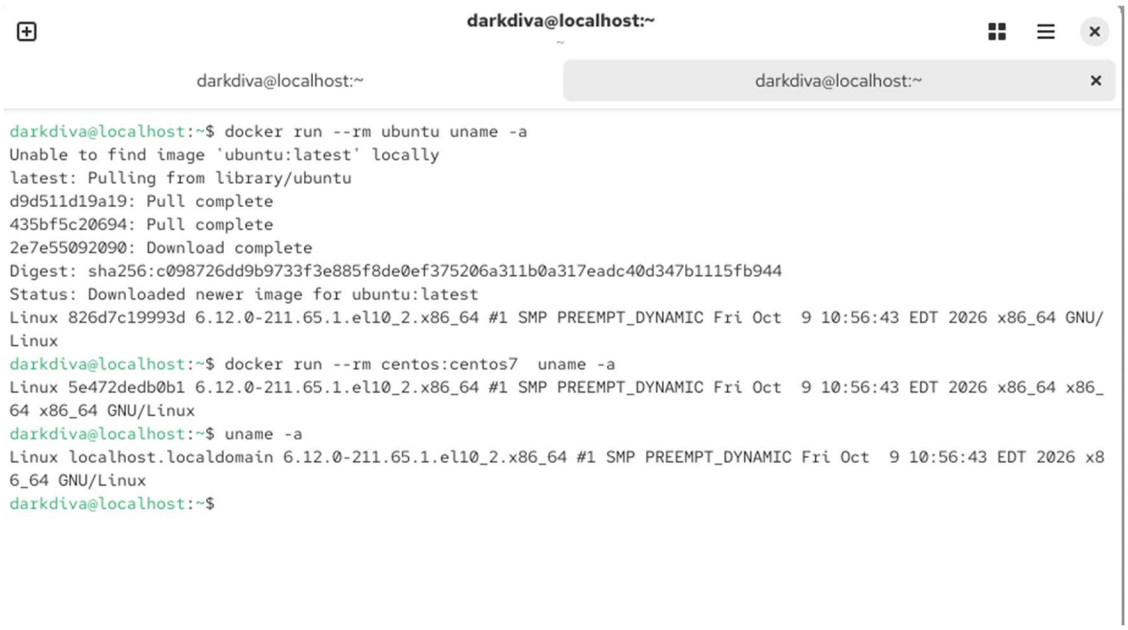


Figure 16 – uname -a in an Ubuntu container, a CentOS container and on the host

Where	Hostname	Kernel
Ubuntu container	826d7c19993d	6.12.0-211.65.1.el10_2.x86_64
CentOS 7 container	5e472dedb0b1	6.12.0-211.65.1.el10_2.x86_64
RHEL host (VM)	localhost.localdomain	6.12.0-211.65.1.el10_2.x86_64

Conclusion. The Ubuntu and CentOS containers display the RHEL kernel of the host (el10), even though their user spaces are Ubuntu and CentOS. Containers therefore share the kernel of their host, unlike virtual machines, which each run their own kernel. Only the hostname differs: it is the container ID, because each container has its own UTS namespace. A container is an isolated process, not a full operating system.

7. The hello-world image

```
docker run hello-world
```

```
darkdiva@localhost:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:11a3eb1597e864c51b3f56733cb2e8a90bf0c284bfc05f9444d81c558bc848aa
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Figure 17 – Output of docker run hello-world

The image was not available locally, so Docker pulled it from Docker Hub first (Unable to find image ... locally), then ran it and displayed Hello from Docker!. According to the message, Docker performed four steps:

- The Docker client contacted the Docker daemon.
- The daemon pulled the hello-world image from Docker Hub (amd64 version).
- The daemon created a new container from that image, which runs the executable that produces the output.
- The daemon streamed this output to the client, which sent it to the terminal.

The image is extremely small (21.8 kB on disk, 9.49 kB compressed): it is built FROM scratch, an empty base, and contains only one compiled program (hello) started with CMD ["/hello"]. The container ran, printed its message and exited with code 0, as shown below.

```
darkdiva@localhost:~$ docker images
                                                                  i Info →  U In Use
IMAGE                                          ID             DISK USAGE   CONTENT SIZE   EXTRA
centos:centos7                                 be65f488b776       299MB         76.1MB
hello-world:latest                             11a3eb1597e8      21.8kB         9.49kB    U
registry.access.redhat.com/ubi9/ubi:latest     16e9e43d99e1       309MB         80.5MB
ubuntu:latest                                  c098726dd9b9       160MB         45.9MB
darkdiva@localhost:~$ docker ps -a
CONTAINER ID   IMAGE         COMMAND    CREATED              STATUS                          PORTS     NAMES
f7795f10d0ab   hello-world   "/hello"   About a minute ago   Exited (0) About a minute ago             naughty_b
org
darkdiva@localhost:~$
```

Figure 18 – Images present and the exited hello-world container

8. Problems encountered and remarks

Observation	Cause	Handling
VM runs RHEL 10, lab targets RHEL 9	Newer release	Same commands worked; only the kernel tag differs (el10)

Observation	Cause	Handling
docker pull centos fails	Tag latest removed, image deprecated	Use an explicit tag: centos:centos7
Storage driver is overlayfs, not overlay2	Docker 29 uses the containerd image store	Normal, no action needed
Containers exit right after start	Default command /bin/bash has no input without -it	Expected behavior; use -it for an interactive shell
Group change not effective at first	Session must be reopened after usermod	Logged out and back in

9. Conclusion

Docker CE 29.9.1 was installed on a RHEL 10 virtual machine and used successfully without sudo. The lab covered the image commands (search, pull, list, remove), the container life cycle (create, start, stop, run, remove, --rm) and the hello-world image. The main lessons are that an image must be pulled with a tag that exists, that a container lives only as long as its main process, and that containers share the kernel of the host while being isolated by namespaces. This last point distinguishes them from virtual machines and explains why they are lighter and faster to start.